Manual de instalación

Preparación del entorno de trabajo del taller

Windows y Ubuntu, paso a paso

MUIA
Máster Universitario en Inteligencia Artificial

Escuela Superior de Informática
Universidad de Castilla-La Mancha
Ciudad Real

3 de octubre de 2026

Todas las versiones, identificadores de paquetes y comandos de este manual han sido verificados contra las fuentes oficiales de cada herramienta.

Índice

Manual de instalación **4**
Preparación del entorno de trabajo del taller 4

1. Python y lo necesario para crear entornos **4**
1.1. Qué versión instalar y por qué 4
1.2. Instalación en Windows 5
1.3. Instalación en Ubuntu 5
1.4. Verificación del apartado 1 5

2. Visual Studio Code y las extensiones de Python y Jupyter **6**
2.1. Instalación del editor 6
2.2. Extensiones necesarias 6
2.3. Abrir la carpeta del taller 6
2.4. Verificación del apartado 2 6

3. Ollama y una cuenta en su nube **7**
3.1. Instalación en Windows 7
3.2. Instalación en Ubuntu 7
3.3. Crear la cuenta en la nube 7
3.4. Iniciar sesión desde el equipo 7
3.5. Clave de API (solo si vas a acceder a la nube sin Ollama instalado) 7
3.6. Descargar un modelo local 8
3.7. Verificación del apartado 3 8

4. Crear el entorno del taller con todas sus librerías **8**
4.1. Una aclaración importante: `.venv` no es lo mismo que `.env` 8
4.2. Situarse en la carpeta del material 8
4.3. Crear el entorno virtual 9
4.4. Instalar las librerías 9
4.5. Registrar el entorno como kernel de Jupyter 9
4.6. Crear el fichero de configuración `.env` 9
4.7. Verificación del apartado 4 10

5. Iniciar el servidor de Ollama y verificar todo **10**
5.1. Cómo se inicia el servidor 10
5.2. Comprobar que responde 10
5.3. Verificación completa del entorno del taller 11
5.4. Ejecutar la primera libreta 11

6. Problemas frecuentes **11**
6.1. «`uv` no se reconoce como un comando» 11
6.2. «No se encuentra el kernel Seminario LangGraph» en VS Code 12
6.3. `ModuleNotFoundError` al ejecutar una libreta 12
6.4. El modelo de nube falla: cuotas y modelos retirados 12
6.5. El modelo no sabe usar herramientas 12
6.6. La descarga del modelo se interrumpe o va muy lenta 13
6.7. El servidor no arranca porque el puerto está ocupado 13

7. Chuleta de comandos **13**
7.1. Windows (PowerShell) 13
7.2. Ubuntu 13
7.3. Lista de comprobación final 13

Manual de instalación

Preparación del entorno de trabajo del taller

Este manual describe, paso a paso y para **Windows** y **Ubuntu**, la instalación de las herramientas necesarias para seguir el taller. Al terminar tendrás un entorno de trabajo reproducible, un editor configurado y acceso a un modelo de lenguaje funcionando.

Tiempo estimado: entre 40 y 60 minutos, según la velocidad de tu conexión. La mayor parte del tiempo se consume descargando el modelo y las librerías.

Qué vamos a instalar

#	Componente	Para qué sirve
1	Python y gestor de entornos	Ejecutar el código y aislar las librerías del sistema
2	Visual Studio Code y sus extensiones	Editar y ejecutar las libretas `.ipynb`
3	Ollama y una cuenta en su nube	Acceder al modelo de lenguaje
4	El entorno del taller (`.venv`) y su configuración (`.env`)	Tener las versiones exactas que usa el material
5	Servidor de Ollama en marcha y verificación final	Comprobar que todo funciona antes de empezar

Antes de empezar

Vas a necesitar:

- Conexión a internet.
- Permisos de administrador en el equipo (para instalar programas).
- Unos 6 GB de espacio libre en disco si descargas un modelo local.
- Una cuenta de correo para registrarte en Ollama.

> **Consejo:** no instales todo de golpe. Ve verificando cada apartado con el comando que se indica al final de cada uno. Si algo falla, es mucho más fácil localizarlo.

---

1 Python y lo necesario para crear entornos

1.1 Qué versión instalar y por qué

El material del taller se ha construido y verificado con **Python 3.12**. Puedes usar 3.11, 3.12 o 3.13. **Evita Python 3.14**: aunque se instala, LangGraph no declara soporte oficial para esa versión y podrías encontrarte con errores difíciles de diagnosticar.

Además de Python, vamos a instalar **`uv`**, un gestor de entornos y paquetes muy rápido. Es la herramienta que usan los comandos de este manual. Si lo prefieres, más adelante se explica la alternativa con las herramientas estándar (`venv` y `pip`).

1.2 Instalación en Windows

Abre **PowerShell** (menú Inicio → escribe `PowerShell` → clic derecho → *Ejecutar como administrador*).

Paso 1. Instalar Python 3.12.

```
winget install --id Python.Python.3.12 -e
```

Si `winget` no está disponible, descarga el instalador desde https://www.python.org/downloads/ y **marca la casilla «Add python.exe to PATH»** en la primera pantalla del instalador. Si no la marcas, el comando `python` no funcionará.

Paso 2. Instalar `uv`.

```
winget install --id astral-sh.uv -e
```

Alternativa, si prefieres el instalador oficial:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Paso 3. Cerrar y volver a abrir PowerShell. Esto es necesario para que los cambios en la variable `PATH` surtan efecto. Sin este paso, los comandos parecerán no existir.

1.3 Instalación en Ubuntu

Abre una terminal (`Ctrl` + `Alt` + `T`).

Paso 1. Actualizar el índice de paquetes e instalar Python 3.12.

```
sudo apt update
sudo apt install -y python3.12 python3.12-venv python3-pip curl git
```

Si tu versión de Ubuntu no incluye `python3.12` en sus repositorios (ocurre en versiones anteriores a 24.04), instala `python3` y `python3-venv` y continúa: `uv` puede descargar Python 3.12 por su cuenta sin tocar el sistema.

```
sudo apt install -y python3 python3-venv python3-pip curl git
```

Paso 2. Instalar `uv`.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Paso 3. Recargar la configuración del shell.

```
source ~/.bashrc
```

Paso 4. (Opcional, recomendado) Que `uv` descargue Python 3.12.

```
uv python install 3.12
```

Esta opción es la más limpia: `uv` gestiona su propia copia de Python y no interfiere con el Python del sistema, que en Ubuntu es necesario para otras herramientas.

1.4 Verificación del apartado 1

En **Windows**:

```
python --version
uv --version
```

En **Ubuntu**:

```
python3.12 --version   # o python3 --version
uv --version
```

Debes ver algo similar a `Python 3.12.x` y `uv 0.12.x`. Si `uv` no se reconoce, cierra y vuelve a abrir la terminal.

---

2 Visual Studio Code y las extensiones de Python y Jupyter

2.1 Instalación del editor

Windows (en PowerShell):

```
winget install --id Microsoft.VisualStudioCode -e
```

También puedes descargarlo desde https://code.visualstudio.com/.

Ubuntu:

```
sudo snap install code --classic
```

La opción recomendada por Microsoft para Ubuntu es instalar el paquete `.deb` desde https://code.visualstudio.com/, pero `snap` funciona correctamente y es más sencillo.

2.2 Extensiones necesarias

Abre Visual Studio Code y pulsa `Ctrl` + `Shift` + `X` para abrir el panel de extensiones. Busca e instala estas dos (la tercera se instala automáticamente con la primera):

Extensión	Identificador	Para qué sirve
Python	`ms-python.python`	Ejecutar y depurar código Python
Jupyter	`ms-toolsai.jupyter`	Abrir y ejecutar libretas `.ipynb`
Pylance	`ms-python.vscode-pylance`	Autocompletado (se instala con Python)

También puedes instalarlas desde la terminal, si prefieres no usar el ratón:

```
code --install-extension ms-python.python
code --install-extension ms-toolsai.jupyter
```

2.3 Abrir la carpeta del taller

Es importante abrir **la carpeta**, no un fichero suelto. Así el editor detecta el entorno `.venv` que crearemos en el apartado 4 y las rutas relativas funcionan.

- Interfaz gráfica: *Archivo* → *Abrir carpeta…* y selecciona la carpeta `seminario`.
- Desde la terminal, situado en la carpeta:

```
code .
```

2.4 Verificación del apartado 2

En Visual Studio Code:

1. Abre cualquier fichero `.py` de la carpeta: deberías ver el código coloreado y el botón **Run** (triángulo) arriba a la derecha.
2. Abre una libreta `.ipynb`: deberías ver las celdas y, arriba a la derecha, un selector de **kernel**.
3. Si el selector de kernel no aparece o dice «Select Kernel», es normal: todavía no hemos creado el entorno. Lo resolveremos en el apartado 4.

---

3 Ollama y una cuenta en su nube

Ollama es la herramienta con la que accederemos al modelo de lenguaje, tanto en local (ejecutándose en tu equipo) como en la nube.

3.1 Instalación en Windows

```
winget install --id Ollama.Ollama -e
```

También puedes descargar el instalador `OllamaSetup.exe` desde https://ollama.com/download.

Al terminar la instalación, **Ollama queda ejecutándose en segundo plano** y verás su icono en el área de notificación (junto al reloj). No necesitas arrancarlo manualmente.

3.2 Instalación en Ubuntu

```
curl -fsSL https://ollama.com/install.sh | sh
```

Este comando instala Ollama y lo registra como **servicio del sistema** (`systemd`), de modo que arranca automáticamente. Comprueba que el servicio está activo:

```
systemctl status ollama
```

3.3 Crear la cuenta en la nube

1. Entra en https://ollama.com y pulsa **Sign up**.
2. Regístrate con tu correo electrónico y verifica la cuenta.
3. Cada persona debe usar **su propia cuenta**: los términos de servicio exigen ser mayor de edad y una cuenta por persona. No compartas credenciales.

El plan gratuito incluye acceso a un conjunto de modelos de nube con una cuota mensual limitada. Es suficiente para seguir el taller.

3.4 Iniciar sesión desde el equipo

Vincula tu equipo con la cuenta. Este paso es el que permite usar modelos de nube a través del servicio local, **sin necesidad de manejar claves de API en el código**:

```
ollama signin
```

Se abrirá el navegador para que autorices el acceso. Vuelve a la terminal cuando termine.

3.5 Clave de API (solo si vas a acceder a la nube sin Ollama instalado)

Si quieres usar los modelos de nube desde Python sin depender del servicio local, necesitas una clave:

1. Entra en https://ollama.com/settings/keys.
2. Crea una clave nueva y cópiala.
3. Guárdala para el fichero `.env` del apartado 4.6. La clave se envía como `Authorization: Bearer ...` y la dirección base es `https://ollama.com/v1`.

Esta vía es **opcional**. Si has hecho `ollama signin`, el taller funciona sin ella.

3.6 Descargar un modelo local

Descarga un modelo que se ejecutará en tu equipo. Es la red de seguridad del taller: si la cuota de la nube se agota o no hay internet, el material sigue funcionando.

```
ollama pull gemma4:e4b
```

Son unos 10 GB, así que puede tardar. Si prefieres empezar con algo más ligero para probar, puedes hacerlo más adelante.

3.7 Verificación del apartado 3

```
ollama --version
ollama list
```

> **Atención:** el comando correcto es `ollama --version`, **con dos guiones**. Escribir `ollama version` da error. Del mismo modo, `ollama whoami` **no existe**: para comprobar que la sesión está iniciada, la prueba fiable es invocar un modelo de nube (apartado 5.3).

En `ollama list` deberías ver el modelo que acabas de descargar.

---

4 Crear el entorno del taller con todas sus librerías

4.1 Una aclaración importante: `.venv` no es lo mismo que `.env`

Antes de continuar conviene deshacer una confusión muy frecuente, porque estos dos nombres se parecen mucho y designan cosas completamente distintas:

	`.venv`	`.env`
Qué es	una **carpeta** con una copia aislada de Python y las librerías	un **fichero de texto** con variables de configuración
Para qué sirve	que las versiones del taller no choquen con las del sistema	indicar qué modelo usar y dónde está Ollama
Qué contiene	ejecutables y paquetes	unas pocas líneas de texto

En este manual crearemos la carpeta `.venv` y el fichero `.env`. Si en tu equipo ya tienes un entorno llamado `.env`, funcionará igual, pero conviene renombrarlo para no confundirlos: es un error que cuesta caro porque **el síntoma no es un fallo, sino un comportamiento extraño** (el sistema usa otro Python o lee la configuración equivocada).

4.2 Situarse en la carpeta del material

Windows (en PowerShell):

```
cd $HOME\Descargas\langGraph\seminario
```

Ubuntu:

```
cd ~/Descargas/langGraph/seminario
```

Ajusta la ruta a donde tengas el material. Comprueba que estás en el sitio correcto:

```
dir        # Windows
ls         # Ubuntu
```

Debes ver `requirements.txt`, `comun.py` y las libretas `00_preflight.ipynb`, etc.

4.3 Crear el entorno virtual

```
uv venv --python 3.12 .venv
```

`uv` localizará Python 3.12 y creará la carpeta `.venv`. Si no lo encuentra, lo descargará automáticamente.

Alternativa sin `uv`, con las herramientas estándar:

```
python3.12 -m venv .venv          # Ubuntu
python -m venv .venv              # Windows
```

4.4 Instalar las librerías

Windows:

```
uv pip install --python .venv\Scripts\python.exe -r requirements.txt
```

Ubuntu:

```
uv pip install --python .venv/bin/python -r requirements.txt
```

La instalación descarga la pila completa: `langgraph`, `langchain`, `langchain-openai`, `pydantic`, `ipykernel` y el resto. Tarda un par de minutos.

Alternativa sin `uv`: activa el entorno y usa `pip`.

```
source .venv/bin/activate           # Ubuntu
.venv\Scripts\Activate.ps1          # Windows (PowerShell)
pip install -r requirements.txt
```

4.5 Registrar el entorno como kernel de Jupyter

Este paso es el que hace que Visual Studio Code pueda ejecutar las libretas con las librerías que acabamos de instalar. Sin él, el editor usará otro Python y las libretas fallarán con errores del tipo `ModuleNotFoundError`.

Windows:

```
.venv\Scripts\python.exe -m ipykernel install --user --name seminario-langgraph --display-name "Seminario
    LangGraph"
```

Ubuntu:

```
.venv/bin/python -m ipykernel install --user --name seminario-langgraph --display-name "Seminario LangGraph"
```

4.6 Crear el fichero de configuración `.env`

El material incluye un fichero de ejemplo. Cópialo:

Windows:

```
Copy-Item .env.example .env
```

Ubuntu:

```
cp .env.example .env
```

Ábrelo con el editor y revisa su contenido. Los valores por defecto funcionan sin cambios; solo debes rellenarlo si quieres forzar un modelo o usar una clave de API:

```
# Si se deja vacío, el sistema prueba varios modelos por orden hasta dar con uno que responda
SEMINARIO_MODELO=

# Dirección del servicio local de Ollama
OLLAMA_HOST=http://localhost:11434

# Solo si accedes a la nube sin el servicio local
OLLAMA_API_KEY=
```

El fichero `.env` contiene credenciales si rellenas la clave de API. **No lo subas nunca a un repositorio público.** El material incluye `.env.example` precisamente para eso.

4.7 Verificación del apartado 4

Windows:

```
.venv\Scripts\python.exe -c "import langgraph, langchain; print('librerias OK')"
```

Ubuntu:

```
.venv/bin/python -c "import langgraph, langchain; print('librerias OK')"
```

Y comprueba que el kernel está registrado:

```
jupyter kernelspec list
```

Debe aparecer `seminario-langgraph`.

En Visual Studio Code, abre una libreta `.ipynb` y pulsa **Select Kernel** arriba a la derecha. Debe aparecer **Seminario LangGraph** en la lista. Selecciónalo.

---

5 Iniciar el servidor de Ollama y verificar todo

5.1 Cómo se inicia el servidor

En Windows: no tienes que hacer nada. El instalador deja Ollama ejecutándose en segundo plano y lo arranca al iniciar sesión. Verás su icono junto al reloj.

En Ubuntu: el instalador lo registra como servicio, que arranca solo.

```
sudo systemctl start ollama     # arrancar si no lo está
systemctl status ollama         # comprobar que está activo
```

En ambos casos, si necesitas arrancarlo manualmente (por ejemplo, para ver los mensajes), abre una terminal y ejecútalo en primer plano:

```
ollama serve
```

Esta terminal queda ocupada: es el servidor. Para trabajar, abre otra. Para detenerlo, pulsa `Ctrl` + `C`.

5.2 Comprobar que responde

```
curl http://localhost:11434/api/tags
```

Windows incluye `curl` desde Windows 10. Si no funciona, abre http://localhost:11434/api/tags en el navegador. En ambos casos deberías ver una lista de modelos en formato JSON. Si obtienes un error de conexión, el servidor no está arrancado.

5.3 Verificación completa del entorno del taller

El material incluye un verificador que comprueba, una por una, todas las capacidades necesarias.

Windows:

```
.venv\Scripts\python.exe verificar_entorno.py
```

Ubuntu:

```
.venv/bin/python verificar_entorno.py
```

Tarda alrededor de un minuto, porque envía una petición real a cada componente. La salida debe ser similar a esta:

```
================================================================================
VERIFICACIÓN DEL ENTORNO · Seminario de agentes con LangGraph
================================================================================

  [OK   ] Daemon de Ollama accesible        24 modelos en http://localhost:11434
  [OK   ] Versiones de la pila              langgraph 1.2.12 · langchain 1.4.3
  [OK   ] Modelo elegido y respondiendo     gemma4:31b-cloud (nube)
  [OK   ] Tool calling (CRÍTICO)            tool_calls: sonda
  [OK   ] Salida estructurada               Receta: Pollo al Limón y Ajo Express
  [OK   ] Memoria de corto plazo            4 mensajes en el checkpoint
  [OK   ] Memoria de largo plazo (Store)    escritura y lectura por namespace
  [OK   ] Interrupción humana               pausa y reanudación con Command(resume=...)
  [OK   ] Persistencia SQLite               langgraph-checkpoint-sqlite disponible

  Todo correcto: el seminario puede empezar.
```

La comprobación marcada como **CRÍTICA** es la más importante: verifica que el modelo sabe usar herramientas. Sin esa capacidad no es posible construir agentes. Si falla, consulta el apartado 6.4.

5.4 Ejecutar la primera libreta

Abre `00_preflight.ipynb` en Visual Studio Code, selecciona el kernel **Seminario LangGraph** y pulsa **Run All**. Deberías ver la auditoría de modelos y la confirmación del llamado a funciones.

Ya está todo listo.

---

6 Problemas frecuentes

Los siguientes problemas están recogidos de la preparación real del taller. Se ordenan por frecuencia.

6.1 «`uv` no se reconoce como un comando»

Cierra la terminal y vuelve a abrirla. Los instaladores modifican la variable `PATH`, pero la terminal en uso conserva la configuración antigua.

En Ubuntu, si el problema persiste:

```
source ~/.bashrc
which uv
```

6.2 «No se encuentra el kernel Seminario LangGraph» en VS Code

1. Comprueba que el kernel está registrado: `jupyter kernelspec list`.
2. Si no aparece, repite el paso 4.5.
3. Recarga la ventana de VS Code: `Ctrl` + `Shift` + `P` → *Developer: Reload Window*.
4. Si sigue sin aparecer, en el selector de kernel elige *Select Another Kernel…* → *Jupyter Kernel…* y localiza el Python de dentro de `.venv` a mano.

6.3 `ModuleNotFoundError` al ejecutar una libreta

El kernel seleccionado no es el del taller. Es el error más habitual y siempre tiene la misma causa: se está usando el Python del sistema en lugar del entorno.

Comprueba dentro de una celda de la libreta:

```
import sys
print(sys.executable)
```

La ruta impresa debe contener `.venv`. Si no es así, vuelve a seleccionar el kernel (apartado 4.7).

6.4 El modelo de nube falla: cuotas y modelos retirados

Los catálogos de modelos **cambian**. Durante la preparación del taller se comprobó que, de 24 modelos disponibles, solo 8 respondían. Los mensajes de error son informativos y cada uno exige una solución distinta:

Mensaje	Significado	Solución
`HTTP 402: not included in your free usage`	El modelo no entra en el plan gratuito	Usa otro modelo, o uno local
`HTTP 410: was retired at ...`	El proveedor retiró el modelo	Cambia de modelo: no hay nada que pagar
`HTTP 429: session usage limit`	Cuota agotada	**Cambia a un modelo local**
`HTTP 401: Unauthorized`	Sesión no iniciada o clave incorrecta	Repite `ollama signin`

> **Importante:** si el problema es de cuota, cambiar a otro modelo de nube **no** soluciona nada, porque la cuota es de tu cuenta, no del modelo. La única salida real es el modelo local.

Para usar el modelo local del apartado 3.6:

Windows:

```
$env:SEMINARIO_MODELO="gemma4:e4b"
.venv\Scripts\python.exe verificar_entorno.py
```

Ubuntu:

```
SEMINARIO_MODELO=gemma4:e4b .venv/bin/python verificar_entorno.py
```

O edita el fichero `.env` y pon `SEMINARIO_MODELO=gemma4:e4b`.

6.5 El modelo no sabe usar herramientas

Si el verificador falla en la comprobación crítica, el modelo elegido no soporta el llamado a funciones. Comprueba las capacidades de un modelo con:

```
ollama show gemma4:e4b
```

En la salida debe aparecer `tools` entre las capacidades. Si no aparece, ese modelo no sirve para el taller. El modelo `gemma4:e4b` (local) y `gemma4:31b-cloud` (nube) están verificados y funcionan.

6.6 La descarga del modelo se interrumpe o va muy lenta

Vuelve a lanzarla: `ollama pull` reanuda la descarga donde se quedó.

```
ollama pull gemma4:e4b
```

6.7 El servidor no arranca porque el puerto está ocupado

Ocurre si ya hay una instancia en ejecución. Comprueba si el puerto responde:

```
curl http://localhost:11434/api/tags
```

Si responde, el servidor ya está funcionando y no necesitas arrancar otro.

---

7 Chuleta de comandos

7.1 Windows (PowerShell)

```
# Entorno
uv venv --python 3.12 .venv
uv pip install --python .venv\Scripts\python.exe -r requirements.txt
.venv\Scripts\python.exe -m ipykernel install --user --name seminario-langgraph --display-name "Seminario
    LangGraph"

# Ollama
ollama --version
ollama list
ollama pull gemma4:e4b
ollama serve

# Verificacion
.venv\Scripts\python.exe verificar_entorno.py
```

7.2 Ubuntu

```
# Entorno
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -r requirements.txt
.venv/bin/python -m ipykernel install --user --name seminario-langgraph --display-name "Seminario LangGraph"

# Ollama
ollama --version
ollama list
ollama pull gemma4:e4b
sudo systemctl status ollama

# Verificacion
.venv/bin/python verificar_entorno.py
```

7.3 Lista de comprobación final

☐ `python --version` (en Ubuntu, `python3 --version`) devuelve 3.11, 3.12 o 3.13.
☐ `uv --version` funciona.

- ☐ Visual Studio Code abre una libreta `.ipynb` y muestra el selector de kernel.
- ☐ `ollama --version` funciona y `ollama list` muestra al menos un modelo.
- ☐ Has iniciado sesión en Ollama con `ollama signin`.
- ☐ Existe la carpeta `.venv` y el fichero `.env`.
- ☐ `jupyter kernelspec list` muestra `seminario-langgraph`.
- ☐ El kernel **Seminario LangGraph** está seleccionado en VS Code.
- ☐ `verificar_entorno.py` termina con «**Todo correcto: el seminario puede empezar**».